```
import java.util.*;
public class Puzzle1 {
   public static void main(String[] args) {
      String goal = "123456780";
      Queue<List<String>> q = new LinkedList<>();
      Set<String> seen = new HashSet<>();
      q.add(Arrays.asList("123046758"));
      seen.add("123046758");
      int[] d = {1, -1, 3, -3};
      while (!q.isEmpty()) {
         List<String> path = q.poll();
         String s = path.get(path.size()-1);
         if (s.equals(goal)) {
            for (String x : path) {
               for (int i = 0; i < 9; i++) {
                  System.out.print(x.charAt(i) + " ");
                  if (i % 3 == 2) System.out.println();  }
               System.out.println();}
            System.out.println("Goal Reached!");
            break;
         }
         int p = s.indexOf('0');
         for (int v : d) {
            int n = p + v;
            if (n >= 0 && n < 9 &&
               !(v == 1 && p % 3 == 2) &&
```

```
            !(v == -1 && p % 3 == 0)) {
            char[] a = s.toCharArray();
            a[p] = a[n];
            a[n] = '0';
            String next = new String(a);
            if (seen.add(next)) {
               List<String> newPath = new ArrayList<>(path);
               newPath.add(next);
               q.add(newPath);
}}}}}}
```

1. Theory

Breadth First Search (BFS):

BFS is an uninformed search technique that explores states level by level.

- It uses a Queue (FIFO).
- It starts from the initial state.
- It explores all possible valid moves.
- It stops when the goal state is reached.
- It finds the shortest solution when all moves have equal cost.

BFS in 8 Puzzle:

The 8-puzzle contains 8 numbered tiles and one blank space in a 3×3 grid. The blank space can move up, down, left or right.

2. Algorithm

1. Start.
2. Define the initial state and goal state.
3. Create a queue and visited set.
4. Insert the initial state into the queue.

5. Remove the front state from the queue.
6. Check whether it is the goal state.
7. Generate all valid moves of the blank space.
8. Add unvisited states to the queue.
9. Repeat until the goal is reached.
10. Display the solution path.
11. Stop.

3. Viva Questions and Answers

Q1. What is BFS?
BFS explores states level by level.

Q2. Is BFS informed or uninformed?
Uninformed search.

Q3. Which data structure is used in BFS?
Queue (FIFO).

Q4. Difference between BFS and DFS?
BFS explores level by level, while DFS explores depth-wise.

Q5. How is BFS used in 8-puzzle?
It finds a path from the initial state to the goal state.

Q6. Why is a visited set used?
To avoid repeating the same states.

Q7. Is BFS complete?
Yes, when the branching factor is finite.

Q8. Does BFS find the shortest path?
Yes, when all moves have equal cost.

Q9. What are the valid moves in 8-puzzle?
Up, Down, Left and Right.

Q10. What are the limitations of BFS?
It requires more memory and time for large problems.